



Universidad de
SanAndrés

**I302 - Aprendizaje Automático
y Aprendizaje Profundo**

**Trabajo Práctico 4:
Aprendizaje No-Supervisado**

Ignacio Carlos Basso

30 de mayo de 2025

Ingeniería en Inteligencia Artificial

1. Clustering de datos

Resumen

En este trabajo se aplicaron técnicas de agrupamiento no supervisado al conjunto de datos `clustering.csv`, compuesto por 4999 muestras multivariadas. Se implementaron desde cero tres algoritmos de clustering: *K-Means*, *Gaussian Mixture Models* (GMM) y DBSCAN. El preprocesamiento incluyó normalización, tratamiento de valores faltantes e identificación de columnas irrelevantes. Para *K-Means* y GMM se utilizó el método del codo junto con métricas como el *log-likelihood* y el *silhouette score* para seleccionar el número óptimo de clusters. En el caso de DBSCAN, se exploraron sistemáticamente combinaciones de hiperparámetros ϵ y `minPts`. Los resultados permitieron comparar el desempeño de los modelos en términos de compacidad y separación entre grupos, identificando patrones subyacentes en los datos sin necesidad de etiquetas, obteniendo en todos los casos una clara segmentación de los datos en 15 clusters significativos, a excepción de *K-Means* al utilizar el método del codo con la segunda derivada, que identificó 3 clusters.

1.1. Introducción

El agrupamiento no supervisado es una técnica fundamental en análisis exploratorio de datos, ya que permite descubrir estructuras latentes sin requerir información previa sobre clases o etiquetas. Este tipo de análisis resulta especialmente valioso cuando se trabaja con grandes volúmenes de datos heterogéneos, como ocurre en el presente caso.

El objetivo general del trabajo es analizar un conjunto de muestras multivariadas mediante técnicas de agrupamiento que puedan identificar patrones naturales en los datos. Para ello, se recurrió a tres enfoques clásicos con fundamentos distintos: uno basado en particiones (*K-Means*), otro en modelos probabilísticos (*Gaussian Mixture Models*), y un tercero en densidad (DBSCAN).

En lugar de utilizar implementaciones de librerías existentes, se desarrollaron desde cero cada uno de los algoritmos, lo que permitió un control detallado sobre su funcionamiento y una mejor comprensión de sus limitaciones. Además, se prestó especial atención a la selección de hiperparámetros, adaptando los criterios de evaluación a las particularidades de cada método.

La entrada del análisis consistió en datos multivariados sin etiquetas, y el objetivo fue agruparlos en función de su similitud interna.

La calidad de las agrupaciones se evaluó tanto cualitativa como cuantitativamente, para determinar qué modelo logra una segmentación más fiel a la estructura del conjunto de datos.

A continuación se presentan los métodos utilizados, el preprocesamiento aplicado, la selección de hiperparámetros y los resultados obtenidos.

1.2. Métodos

1.2.1. K-Means

K-Means es un algoritmo de aprendizaje no supervisado que busca dividir un conjunto de datos en k grupos o clusters, minimizando la variabilidad intra-cluster. El criterio de agrupamiento se basa en la distancia euclidiana entre los puntos de datos y los centroides de los clusters. El objetivo del algoritmo es encontrar la partición que minimice la suma de las distancias cuadradas entre los puntos y los centroides del cluster al que pertenecen.

Funcionamiento del algoritmo El algoritmo de *K-Means* se implementó con los siguientes pasos:

1. **Inicialización:** Se eligen aleatoriamente k puntos del conjunto de datos como centroides iniciales.

2. **Asignación:** Cada punto se asigna al cluster cuyo centroide esté más cercano, usando la distancia euclidiana.
3. **Actualización:** Se recalculan los centroides como la media de los puntos asignados a cada cluster.
4. **Repetición:** Se repiten los pasos 2 y 3 hasta que los centroides no cambien significativamente o se alcance un número máximo de iteraciones.
4. Se repiten los pasos 2 y 3 hasta que la función de verosimilitud logarítmica (*log-likelihood*) converge.

1.2.3. DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) es un algoritmo de agrupamiento basado en densidad. A diferencia de *K-Means* o GMM, no requiere especificar el número de clusters y es capaz de identificar clusters con formas arbitrarias, así como puntos atípicos o ruido.

El algoritmo se basa en la idea de regiones densamente conectadas, y se guía por dos hiperparámetros clave: el radio de vecindad ϵ y el número mínimo de puntos *minPts* necesarios para formar un cluster.

El procedimiento general es el siguiente:

1.2.2. Gaussian Mixture Models (GMM)

Los Modelos de Mezcla Gaussiana (GMM) son una extensión probabilística del algoritmo *K-Means*. En lugar de asignar cada punto de datos de forma determinista a un único cluster, GMM asocia a cada punto una probabilidad de pertenecer a cada uno de los clusters, que se modelan como distribuciones gaussianas multivariadas.

El algoritmo GMM se basa en el método de máxima verosimilitud y se implementa mediante el algoritmo EM (*Expectation-Maximization*), el cual alterna entre dos pasos:

1. **Inicialización:** Se inicializan las medias, las matrices de covarianza y los pesos de cada componente. En este trabajo, se utilizó una corrida de *K-Means* para obtener una estimación inicial de las medias.
2. **E-step (*Expectation*):** Se calcula la responsabilidad de cada componente sobre cada dato, es decir, la probabilidad de que ese dato haya sido generado por dicha gaussiana.
3. **M-step (*Maximization*):** Se actualizan las medias, covarianzas y pesos de las gaussianas, ponderando cada dato por su responsabilidad.
3. **Clasificación:** Se distinguen tres tipos de puntos:
 - **Puntos núcleo (*core points*):** tienen al menos *minPts* vecinos dentro del radio ϵ , y pueden expandir un cluster.
 - **Puntos frontera (*border points*):** tienen menos de *minPts* vecinos, pero están dentro del vecindario de un punto núcleo, por lo que se asignan a su mismo cluster.
 - **Ruido (*noise*):** puntos que no cumplen ninguna de las condiciones anteriores, y quedan sin asignar a ningún cluster.

4. Se repiten los pasos hasta que todos los puntos estén clasificados.

En este trabajo, se implementó DBSCAN desde cero, desarrollando una clase `DBSCAN` con métodos auxiliares para la búsqueda de vecinos (`_region_query`) y expansión de clusters. Para mantener la eficiencia en la expansión, se utilizó la estructura `deque`.

1.2.4. Método del codo

El método del codo se utiliza para estimar el número adecuado de grupos al aplicar técnicas de agrupamiento como *K-means*. Consiste en calcular la suma de los errores cuadráticos dentro de los grupos (WCSS) para distintos valores de k , y seleccionar aquel donde la mejora comience a ser marginal. Este punto de inflexión, o “codo” en la curva WCSS vs. k , sugiere un equilibrio entre complejidad y ajuste del modelo.

1.2.5. Silhouette Score

El *silhouette score* es una métrica que evalúa la calidad de una agrupación al comparar cuán similar es cada muestra a las de su propio grupo frente a las de otros grupos. Su valor varía entre -1 y 1 : valores cercanos a 1 indican que las muestras están bien agrupadas, mientras que valores cercanos a 0 o negativos sugieren solapamiento entre grupos o asignaciones incorrectas.

1.3. Descripción del conjunto de datos y preprocesamiento

En un inicio se realizó un análisis exploratorio del conjunto de datos `clustering.csv`, que contiene 4999 muestras. Para ello, se realizó un gráfico de dispersión de los *features* para poder observar la distribución de los datos y tener una intuición sobre la cantidad de clusters que podrían existir. Al observar la Figura 5, se puede apreciar que la distribución de los datos es demasiado compleja como para

poder dar una estimación precisa de la cantidad de clusters que podrían existir. Por esto mismo, es importante la utilización de algoritmos de *clustering* para determinar la cantidad de clusters que existen en el conjunto de datos.

Antes de aplicar los algoritmos de *clustering*, se realizó un preprocesamiento sobre los datos con el objetivo de asegurar su calidad y consistencia. Las operaciones realizadas fueron las siguientes:

1. **Normalización:** Previo al *clustering*, los datos se normalizaron restando la media y dividiendo por el desvío estándar de cada variable, para evitar que las magnitudes afecten el agrupamiento.
2. **Eliminación de la primera columna:** Dado que la primera columna correspondía a identificadores o índices sin valor informativo para el análisis, fue eliminada del conjunto de datos.
3. **Tratamiento de valores faltantes (NaNs):** Se identificaron valores faltantes en distintas columnas del conjunto de datos. Para cada columna con valores ausentes, se procedió a imputarlos utilizando el promedio de los valores no faltantes de esa misma columna.

1.4. Selección de hiperparámetros

Para cada uno de los métodos de *clustering* utilizados (*K-means*, GMM y DBSCAN), se realizó un procedimiento específico de selección de hiperparámetros con el fin de obtener resultados óptimos y comparables. A continuación se detalla el criterio adoptado en cada caso:

1.4.1. K-means

Para determinar el número óptimo de clusters K , se utilizó el **método del codo** (1.2.4),

evaluando la suma total de distancias euclidianas entre los puntos y sus centroides asignados. Se exploraron valores de K en el rango $[1, 20]$, y para cada uno se realizaron múltiples ejecuciones (por defecto, 5) con diferentes inicializaciones aleatorias para mitigar la sensibilidad del algoritmo *K-Means* a mínimos locales, seleccionando luego la ejecución con menor distancia total.

Posteriormente, se calculó la segunda derivada discreta de la curva de distancias. Los valores de K con mayor curvatura fueron considerados como posibles candidatos, seleccionándose finalmente aquel con la derivada de mayor magnitud, lo que indica un cambio abrupto en la ganancia de compactación (saturación del codo).

1.4.2. Gaussian Mixture Models (GMM)

La selección del número óptimo de componentes K se llevó a cabo de manera similar al caso de *K-means*, pero utilizando como métrica el *log-likelihood* obtenido al ajustar el modelo GMM. Se consideraron valores de K en el rango de $[1, 20]$, repitiendo cada ajuste 3 veces para obtener el mejor resultado.

Se graficó el *log-likelihood* en función de K y se calculó su segunda derivada para detectar puntos de inflexión. El valor de K asociado a la mayor curvatura se consideró el más adecuado, ya que representa el punto a partir del cual el aumento de complejidad del modelo no se traduce en una mejora significativa del ajuste.

1.4.3. DBSCAN

La selección de hiperparámetros para el algoritmo DBSCAN —la distancia máxima ε y el número mínimo de puntos minPts — se realizó mediante un **grid search**, evaluando múltiples combinaciones posibles sobre una grilla de valores cuidadosamente construida.

Grid de búsqueda. El rango de valores para ε se generó a partir de los percentiles bajos (1 % a 40 %) de las distancias euclidianas entre puntos seleccionados aleatoriamente del conjunto de datos, complementado con valores comunes predefinidos (0.5 a 3.0). Para minPts , se aplicó la heurística $\text{minPts} \geq D + 1$, con un rango extendido hasta $D + 15$, en pasos de 2.

Evaluación de combinaciones. Cada combinación $(\varepsilon, \text{minPts})$ fue evaluada ejecutando DBSCAN y calculando:

- Número de clusters (excluyendo ruido),
- Proporción de puntos considerados ruido,
- *Silhouette score*, ajustado para excluir puntos ruidosos.

Selección óptima. Se definió una métrica compuesta para balancear calidad de clustering, proporción de ruido y número de clusters:

$$\text{score} = \text{silhouette} \times (1 - \text{ratio_ruido}) \times \log_2(\text{n_clusters} + 1)$$

Esta métrica penaliza configuraciones con demasiados puntos ruidosos y premia aquellas con mayor calidad de clustering (según *silhouette*) y mayor número de clusters (controlado logarítmicamente para evitar sobresegmentación).

La mejor configuración fue la que maximizó este *score*, y se reportaron las tres combinaciones más destacadas.

Este *grid search* automatizado permitió una selección robusta de hiperparámetros, adaptada a la estructura del conjunto de datos y basada en métricas internas de calidad de *clustering*.

1.5. Resultados

Se comenzó el análisis aplicando el algoritmo de *K-Means* (1.2.1) al conjunto de datos previamente preprocesado, con el objetivo de determinar el número óptimo de clusters. Utilizando el método del codo, se identificó un valor óptimo de $K = 3$, como se muestra en la Figura 1. Este punto corresponde al inicio de la estabilización en la disminución de la inercia, es decir, de la suma de distancias al centroide.

El método del codo es una heurística que busca identificar un punto en el cual aumentar el número de clusters deja de producir mejoras significativas en la compactación de los grupos. Sin embargo, al tratarse de una técnica empírica basada en la observación de la curva de inercia, no siempre refleja con precisión la estructura subyacente de los datos. En particular, el algoritmo *K-Means* asume que los clusters son simétricos y bien separados, lo cual no se cumple cuando los grupos tienen formas alargadas, densidades variables, o están muy próximos entre sí. Como resultado, tiende a fusionar algunos clusters o subestimar su número. Por lo tanto, es esperable que otros métodos, como GMM o DBSCAN, identifiquen una mayor cantidad de agrupaciones al modelar mejor la forma y distribución de los datos.

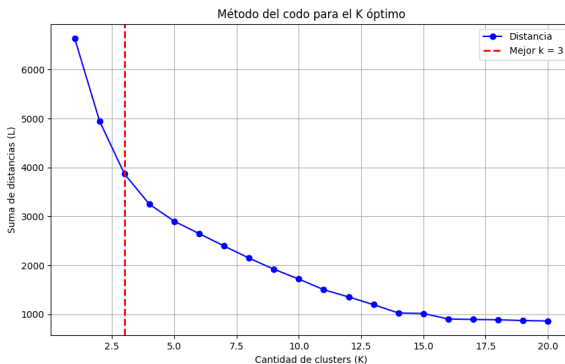


Figura 1: Método del codo para determinar el número óptimo de clusters en K-Means.

Para contrastar la estimación obtenida

con el método del codo, se evaluó el número de clusters utilizando el coeficiente de *Silhouette* (1.2.5), el cual ofrece una medida más robusta de la calidad del agrupamiento. Esta métrica evalúa al mismo tiempo la compactidad interna y la separación entre grupos, lo que la hace útil cuando los datos no son uniformes ni simétricos.

El valor óptimo de K según el coeficiente de *Silhouette* resultó ser $K = 15$, como se observa en la Figura 7. La correspondiente visualización de los clusters generados (Figura 8) muestra una estructura más coherente con la dispersión observada en los datos originales, lo cual sugiere que esta cantidad de agrupaciones representa de forma más precisa la organización latente de los datos.

Posteriormente, al aplicar el algoritmo de *Gaussian Mixture Models* (GMM) (1.2.2), se obtuvo un valor óptimo de $K = 15$ clusters, como se observa en la Figura 2. Este resultado es más acorde con la intuición inicial del análisis, ya que GMM permite una mayor flexibilidad al modelar clusters con diferentes formas, orientaciones y varianzas. La Figura 9 muestra los clusters obtenidos.

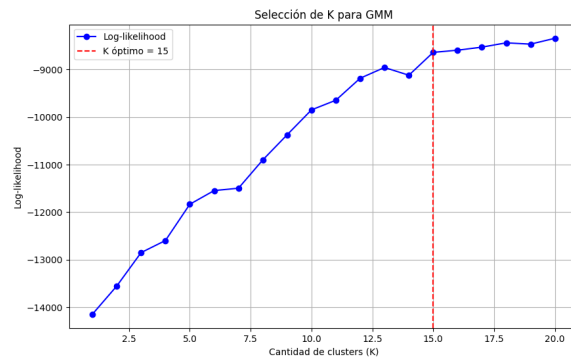


Figura 2: Gráfico del log-likelihood en función del número de componentes K para GMM.

La selección del número óptimo de componentes K se llevó a cabo de manera similar al caso de *K-means*, pero utilizando como métrica el *log-likelihood* obtenido al ajustar el modelo de Mezclas de Gaussianas (GMM).

En la Figura 2 se muestra el valor del *log-*

likelihood en función de la cantidad de clusters K . Se observa una tendencia creciente, lo que indica que el ajuste del modelo mejora a medida que se incrementa K . Sin embargo, a partir de cierto punto esta mejora se vuelve marginal.

Para determinar el valor óptimo de K , se calculó la segunda derivada del *log-likelihood* con respecto a K , con el objetivo de detectar puntos de inflexión. El valor de K correspondiente a la mayor curvatura se considera el más adecuado, ya que representa el punto a partir del cual el aumento de complejidad del modelo no se traduce en una mejora significativa del ajuste.

Según este criterio, el valor óptimo hallado fue $K = 15$, lo cual está marcado en el gráfico mediante una línea punteada roja. Este valor coincide con una desaceleración clara en el crecimiento del *log-likelihood*, validando el uso de este enfoque para seleccionar el número de componentes del modelo.

Finalmente, al aplicar el algoritmo DBSCAN (1.2.3), se utilizó un enfoque automatizado para seleccionar los hiperparámetros óptimos. Se determinó que los mejores valores eran $\epsilon = 0.096$ y $\text{minPts} = 17$ (obtenido del top que se puede encontrar en el cuadro 1), lo que resultó en la identificación de 15 clusters, un 18.98 % de puntos clasificados como ruido y un $\text{score} = 2.38$, con un *Silhouette score* = 0.735. La Figura 10 muestra los clusters obtenidos. Como se puede apreciar DBSCAN también identifica correctamente los 15 clusters, dado que estos presentan alta densidad local y están separados por regiones de baja densidad.

En contraste con algoritmos como *K-Means* y GMM, que tienden a generar clus-

ters con formas regulares y requieren suposiciones sobre la distribución de los datos, DBSCAN ofreció una mayor adaptabilidad a configuraciones complejas. Su criterio de agrupamiento por densidad le permitió detectar clusters no convexos y manejar adecuadamente zonas con diferentes concentraciones de puntos. Además, su capacidad para ignorar automáticamente los puntos aislados lo hace especialmente eficaz en la presencia de ruido o valores atípicos, sin forzar una asignación artificial a algún grupo.

#	ϵ	minPts	Clusters	Ruido	Score
1	0.096	17	15	18.98 %	2.38
2	0.096	9	14	7.50 %	2.20
3	0.096	13	13	13.62 %	2.19

Cuadro 1: Top 3 configuraciones de hiperparámetros para DBSCAN. Las puntuaciones de *Silhouette* fueron 0.73, 0.61 y 0.67 respectivamente.

En conclusión, el bajo rendimiento de *K-Means* se debe a sus fuertes supuestos de esfericidad y separación uniforme entre clusters, lo que lo vuelve inadecuado para datos con formas complejas o densidades variables. Por el contrario, GMM y DBSCAN, al modelar explícitamente la forma y la densidad de los grupos, lograron una segmentación más coherente con la estructura real de los datos, confirmando su adecuación para este conjunto en particular. Sumado a ello, DBSCAN demostró ser especialmente útil para datos con formas no lineales y clusters de densidades variables, gracias a su enfoque basado en densidad que evita imponer una partición rígida del espacio, lo cual lo posiciona como el mejor modelo para este conjunto de datos.

2. Reducción de dimensionalidad

Resumen

En este trabajo se abordó el problema de la alta dimensionalidad en conjuntos de datos mediante técnicas de reducción de dimensionalidad. Se utilizaron imágenes del conjunto de datos `MNIST_dataset.csv`, representadas como vectores de 784 dimensiones, con

el objetivo de obtener representaciones más compactas que conserven la mayor cantidad posible de información. Para ello, se implementaron y compararon dos enfoques: *Principal Component Analysis* (PCA), de naturaleza lineal y estadística, y *Variational Autoencoder* (VAE), un modelo generativo basado en redes neuronales. Se evaluó el rendimiento de ambos métodos en términos de calidad de reconstrucción, utilizando el error cuadrático medio (ECM) como métrica principal. Los resultados muestran que PCA permite conservar una gran parte de la varianza con relativamente pocos componentes, mientras que el VAE logra reconstrucciones de buena calidad a partir de un espacio latente de baja dimensión, destacándose por su capacidad de capturar estructuras no lineales en los datos.

2.1. Introducción

En muchos contextos del aprendizaje automático y la ciencia de datos, los conjuntos de datos poseen una alta dimensionalidad que dificulta tanto su análisis como su visualización, y puede impactar negativamente en el rendimiento de los modelos. Este problema se conoce como la *maldición de la dimensionalidad*, y motiva el uso de técnicas que permitan reducir la cantidad de variables manteniendo la mayor cantidad posible de información.

En este trabajo se estudian y comparan dos enfoques para la reducción de dimensionalidad: uno lineal (PCA), basado en técnicas estadísticas tradicionales, y otro no lineal (VAE), basado en redes neuronales profundas. Para ello, se utilizaron imágenes del conjunto de datos `MNIST_dataset.csv`, donde cada muestra es una imagen en escala de grises de tamaño 28×28 píxeles, representada como un vector de 784 dimensiones.

El objetivo principal es obtener una representación de menor dimensión que permita reconstruir de forma precisa las imágenes originales. Por lo tanto, la entrada del algoritmo consiste en imágenes de dígitos manuscritos normalizadas, mientras que la salida es una versión reconstruida de dichas imágenes. Para evaluar el rendimiento de los métodos, se mide la calidad de reconstrucción mediante el error cuadrático medio (ECM), y se analiza la relación entre la cantidad de dimensiones retenidas y la varianza explicada o la fidelidad de la reconstrucción.

De esta forma, se busca determinar cuántas dimensiones son necesarias para re-

presentar adecuadamente las imágenes, y explorar las ventajas y limitaciones de cada enfoque aplicado sobre este conjunto de datos.

2.2. Métodos

2.2.1. Principal Component Analysis (PCA)

El Análisis de Componentes Principales (PCA) es una técnica de reducción de dimensionalidad que busca encontrar una nueva base ortonormal sobre la cual los datos puedan proyectarse, preservando la mayor cantidad posible de varianza. Dado un conjunto de datos centrado, PCA consiste en descomponer la matriz de datos mediante descomposición en valores singulares (SVD):

$$X = USV^T \quad (1)$$

donde las filas de V (también llamadas *componentes principales*) definen las direcciones ortogonales en las que los datos tienen mayor varianza. Para reducir la dimensionalidad, se seleccionan las primeras k filas de V , proyectando los datos centrados sobre este subespacio:

$$X_k = XV_k^T \quad (2)$$

y reconstruyendo luego una aproximación de los datos originales:

$$X_{\text{rec}} = X_k V_k \quad (3)$$

La calidad de la reconstrucción se evalúa comparando las matrices original y reconstruida, y la varianza explicada por cada compo-

nente se calcula a partir de los valores singulares S_i :

$$\text{Var}(i) = \frac{S_i^2}{n-1} \quad (4)$$

donde n es el número de observaciones. A partir de esto, se construye el cociente de varianza explicada acumulada, útil para seleccionar la cantidad de componentes necesarias para capturar un cierto porcentaje de la información del conjunto original.

2.2.2. Variational Autoencoder (VAE)

El *Variational Autoencoder* (VAE) es un modelo generativo basado en redes neuronales que permite aprender una representación latente continua y compacta de los datos. A diferencia del Autoencoder clásico, el VAE impone una estructura probabilística sobre el espacio latente, asumiendo que las variables latentes $\mathbf{z}$ siguen una distribución normal estándar.

Dado un conjunto de datos $\mathbf{x} \in [0, 1]^D$, el modelo aprende dos redes: un codificador que estima los parámetros de una distribución gaussiana $q_\phi(\mathbf{z}|\mathbf{x})$ y un decodificador que modela $p_\theta(\mathbf{x}|\mathbf{z})$. El entrenamiento maximiza la *Evidence Lower Bound* (ELBO), que puede expresarse como:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = E_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \quad (5)$$

donde el primer término corresponde a la reconstrucción y el segundo a la divergencia de Kullback–Leibler que regula la distribución latente hacia una normal estándar $\mathcal{N}(0, I)$.

Para permitir la retropropagación a través de la muestra latente, se utiliza el *truco de reparametrización*:

$$\mathbf{z} = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (6)$$

El modelo entrenado consiste en un codificador y un decodificador con una capa oculta

intermedia, y un espacio latente de dimensión $d = 20$. Se utilizó la función de pérdida:

$$\mathcal{L} = \text{BCE}(\mathbf{x}, \hat{\mathbf{x}}) + D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2)||\mathcal{N}(0, 1)) \quad (7)$$

donde la *Binary Cross Entropy* se usa como error de reconstrucción, ya que los datos fueron normalizados en el intervalo $[0, 1]$. El modelo fue entrenado con descenso por gradiente usando Adam durante 100 épocas.

2.2.3. Descripción del conjunto de datos y preprocesamiento

Se utilizó el conjunto de datos `MNIST_dataset.csv`, una versión tabular del dataset MNIST con imágenes de dígitos manuscritos representadas como vectores de 784 dimensiones (28×28 píxeles). Por ello mismo, para comenzar se realizó un análisis exploratorio de los datos, observando la distribución de las clases, que se muestra en la Figura 3. De esta manera, se puede observar que el conjunto de datos está balanceado, con 70.000 muestras distribuidas uniformemente entre las 10 clases (dígitos del 0 al 9).

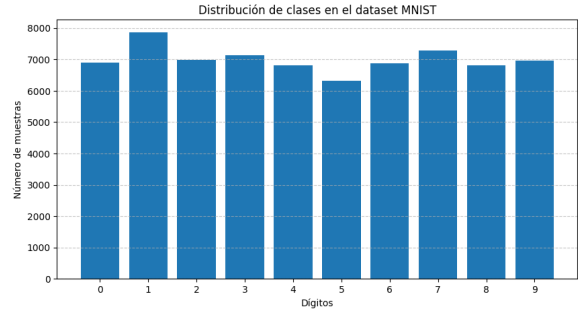


Figura 3: Distribución de clases en el conjunto de datos MNIST.

Posteriormente, cada muestra se normalizó dividiendo los valores por 255 para llevarlos al rango $[0, 1]$.

Para el último inciso, en el que se comparará PCA con VAE el conjunto de datos se divide en subconjuntos de entrenamiento y

validación utilizando una proporción de 80 % y 20 %, respectivamente. Esta partición es estratificada de acuerdo a las etiquetas, con el fin de preservar la distribución de clases en ambos subconjuntos.

2.2.4. Hiperparámetros y validación

El *Autoencoder Variacional* (VAE) se entrenó con los siguientes hiperparámetros: dimensión latente $d_z = 20$, capa oculta de 400 unidades con activación ReLU, función de pérdida compuesta por *binary cross entropy* más divergencia KL, optimizador Adam ($\eta = 10^{-3}$), batch size de 128 y 100 épocas. La selección de hiperparámetros se realizó mediante experimentación empírica, evaluando el error de validación por época. No se aplicó *early stopping*, dado que no se evidenció sobreajuste.

2.2.5. Métricas de rendimiento

Para evaluar la calidad de las reconstrucciones generadas por los modelos de reducción de dimensionalidad (PCA y VAE), se utilizó como métrica principal el **error cuadrático medio** (ECM), definido como:

$$\text{ECM} = \frac{1}{n \cdot d} \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (8)$$

donde $\mathbf{x}_i$ representa la imagen original, $\hat{\mathbf{x}}_i$ su reconstrucción, d la dimensión del espacio de entrada (784 para MNIST) y n el número de muestras evaluadas. Esta métrica cuantifica la pérdida de información entre la imagen original y la reconstruida.

En el caso específico del VAE, además del ECM, se monitoreó la **función de pérdida total** compuesta por dos términos:

- **Pérdida de reconstrucción**, medida mediante *binary cross entropy*, que penaliza la diferencia entre la imagen original y la reconstruida, asumiendo que los píxeles siguen una distribución Bernoulli.

- **Divergencia KL** entre la distribución codificada $q_\phi(\mathbf{z}|\mathbf{x})$ y la distribución prior $p(\mathbf{z})$, que actúa como regularizador para mantener el espacio latente estructurado.

El monitoreo de la pérdida total sobre el conjunto de validación permitió evaluar el desempeño general del VAE durante el entrenamiento, y detectar posibles signos de sobreajuste.

2.3. Resultados

En una primera etapa, se realizó un análisis de componentes principales (PCA) con el objetivo de evaluar cuántos componentes son necesarios para reconstruir las imágenes con un error bajo, así como determinar la cantidad de varianza que se conserva en función del número de componentes retenidos.

En la Figura 4 se muestra la evolución del error cuadrático medio (ECM) en función de la cantidad de componentes utilizados. Se observa que el ECM disminuye rápidamente al incrementar el número de componentes, pero comienza a estabilizarse alrededor de los 154 componentes, alcanzando un valor cercano a 0.003. Esto sugiere que agregar más componentes a partir de ese punto aporta escasa mejora en la calidad de la reconstrucción.

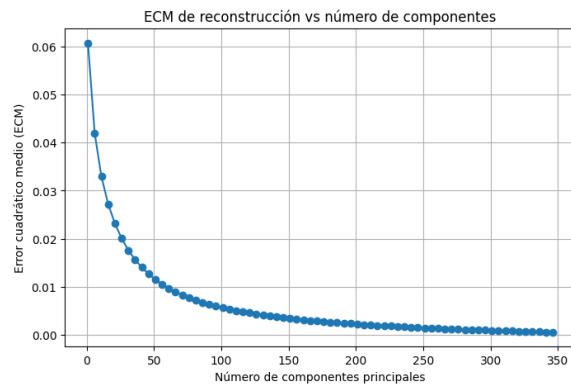


Figura 4: Error cuadrático medio (ECM) en función del número de componentes retenidos en PCA.

Además, se analizó la varianza explicada acumulada por los componentes principales, como se presenta en la Figura 11. Para ilustrar el impacto visual de distintos niveles de varianza retenida, se reconstruyeron imágenes con umbrales del 75 %, 80 %, 85 %, 90 %, 95 % y 99 %, correspondientes a las Figuras 12, 13, 14, 15, 16 y 17.

Las reconstrucciones con un 75 % de varianza retenida presentan similitud general con las imágenes originales, aunque con distorsiones visibles. A medida que se incrementa el umbral de varianza, las imágenes reconstruidas se vuelven progresivamente más fieles. Con un 80 % de varianza (43 componentes), las reconstrucciones ya muestran una alta calidad visual, lo cual indica que PCA logra capturar la estructura subyacente de los datos de forma efectiva.

La Tabla 2 resume la relación entre el umbral de varianza retenida, la cantidad de componentes necesarios y el ECM correspondiente. Se observa una mejora continua del error de reconstrucción conforme se retiene mayor varianza.

Umbral de var.	Componentes	ECM
75 %	33	0.0152
80 %	43	0.0119
85 %	59	0.0087
90 %	87	0.0056
95 %	154	0.0027
99 %	331	0.0005

Cuadro 2: Comparación entre umbrales de varianza retenida y el ECM de reconstrucción en PCA.

Luego de seleccionar un umbral de varianza del 80 %, se reconstruyeron 10 imágenes del conjunto de validación utilizando PCA. En la Figura 18 se presentan las imágenes originales junto con sus respectivas reconstrucciones. Se confirma así la calidad observada previamente: los dígitos son claramente reconocibles y las distorsiones son mínimas. El ECM obtenido en este caso fue de 0.012, reafirmando que

PCA, incluso con un número reducido de componentes, es capaz de lograr reconstrucciones visualmente precisas y eficientes.

Para finalizar, se entrenó un Autoencoder Variacional (VAE) utilizando la librería *PyTorch* sobre el conjunto de datos de imágenes de dígitos manuscritos. Dicho conjunto fue dividido en subconjuntos de entrenamiento (80 %) y validación (20 %), empleando el primero para ajustar los pesos del modelo y el segundo para evaluar su desempeño y afinar los hiperparámetros.

En la Figura 19 se presentan 10 imágenes del conjunto de validación junto con sus respectivas reconstrucciones generadas por el VAE. El modelo logró un error cuadrático medio (ECM) de 0.010, y las reconstrucciones muestran una alta fidelidad visual respecto a las imágenes originales, preservando las formas características de los dígitos manuscritos, aunque con ligeras distorsiones en algunos casos.

Para comparar este enfoque con una técnica lineal de reducción de dimensionalidad, se aplicó PCA sobre el mismo conjunto de datos, reteniendo un 80 % de la varianza (43 componentes). En la Figura 20 se muestran las reconstrucciones generadas por PCA, las cuales presentan un ECM de 0.012. Si bien el error es comparable al del VAE, las imágenes reconstruidas mediante PCA tienden a ser más borrosas y pierden detalles finos, lo que refleja la naturaleza lineal del método.

En resumen, tanto PCA como VAE logran reconstruir las imágenes con bajo error, siendo PCA más eficiente en términos computacionales y de almacenamiento. Sin embargo, el VAE ofrece una mayor capacidad de representación al capturar estructuras más complejas de los datos y, además, permite la generación de nuevas muestras a partir del espacio latente aprendido —una funcionalidad ausente en PCA. Por lo tanto, el VAE se posiciona como una herramienta más flexible y poderosa para tareas que requieren generación de datos o modelado probabilístico.

A. Apéndice

A.1. Distribución de los datos

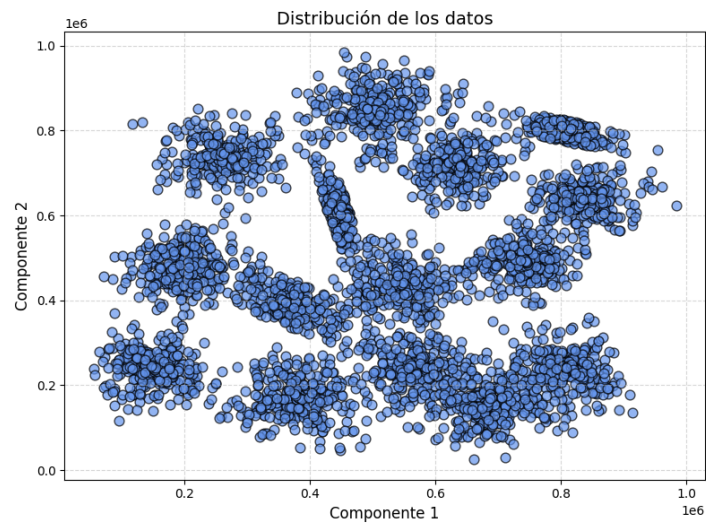


Figura 5: Distribución del *feature* A del conjunto de datos `clustering.csv`.

A.2. K-means clustering

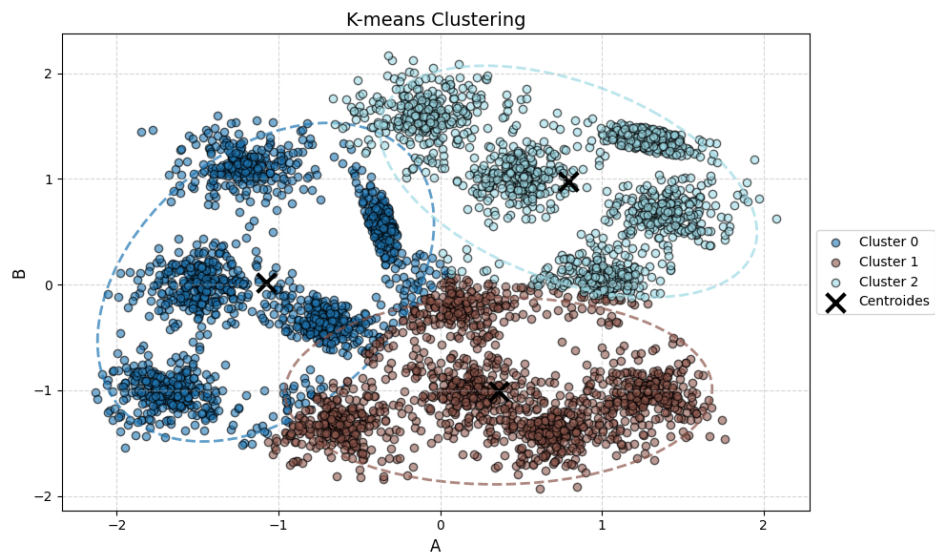


Figura 6: Clusters obtenidos con K-Means. Cada color representa un cluster distinto.

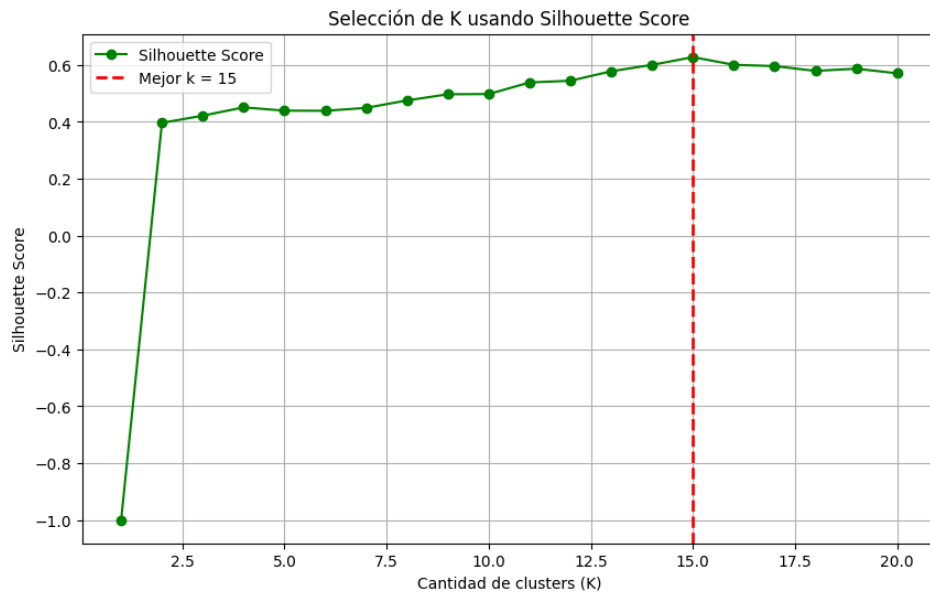


Figura 7: Visualización del coeficiente de Silhouette para cada punto en el espacio de características. Los valores cercanos a 1 indican una buena asignación al cluster, mientras que los valores cercanos a -1 sugieren que el punto podría estar mal clasificado.

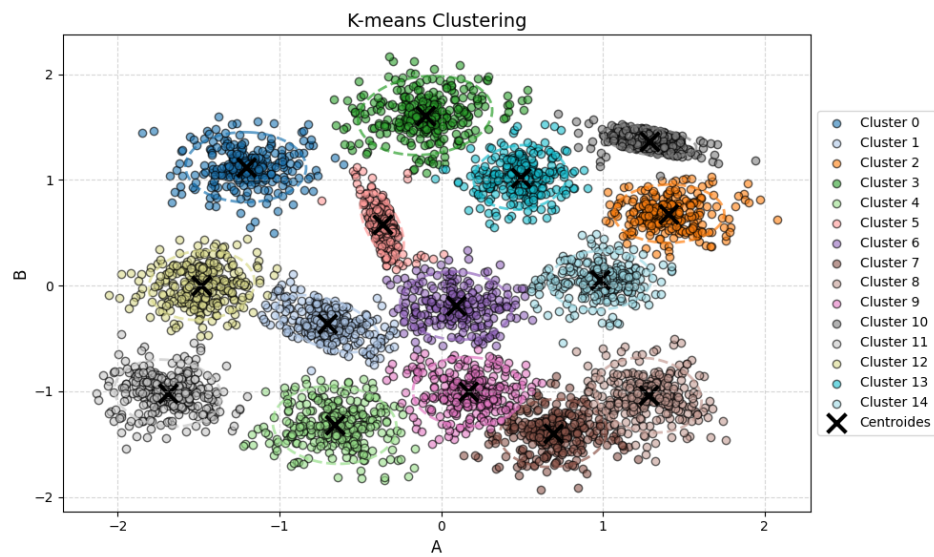


Figura 8: Clusters obtenidos con K-Means aplicando Silhouette. Cada color representa un cluster distinto.

A.3. GMM clustering

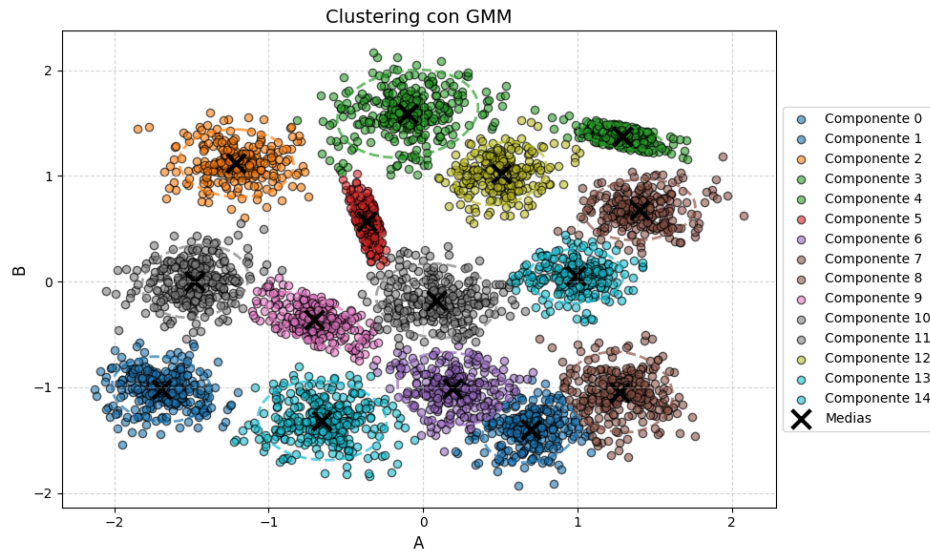


Figura 9: Visualización de los clusters obtenidos con GMM. Cada color representa una distribución gaussiana distinta.

A.4. DBSCAN clustering

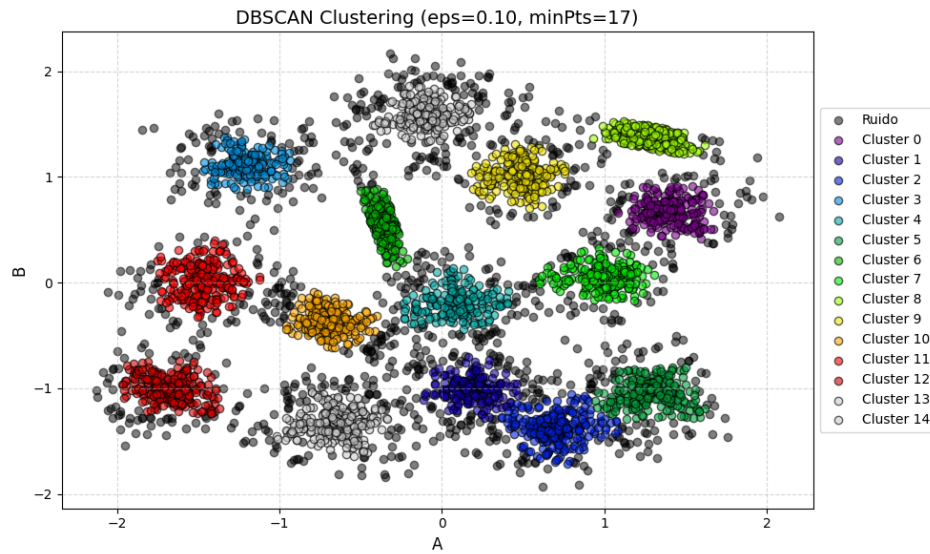


Figura 10: Visualización de los clusters obtenidos con DBSCAN. Los puntos en gris representan el ruido.

A.5. Varianza acumulada al aplicar PCA

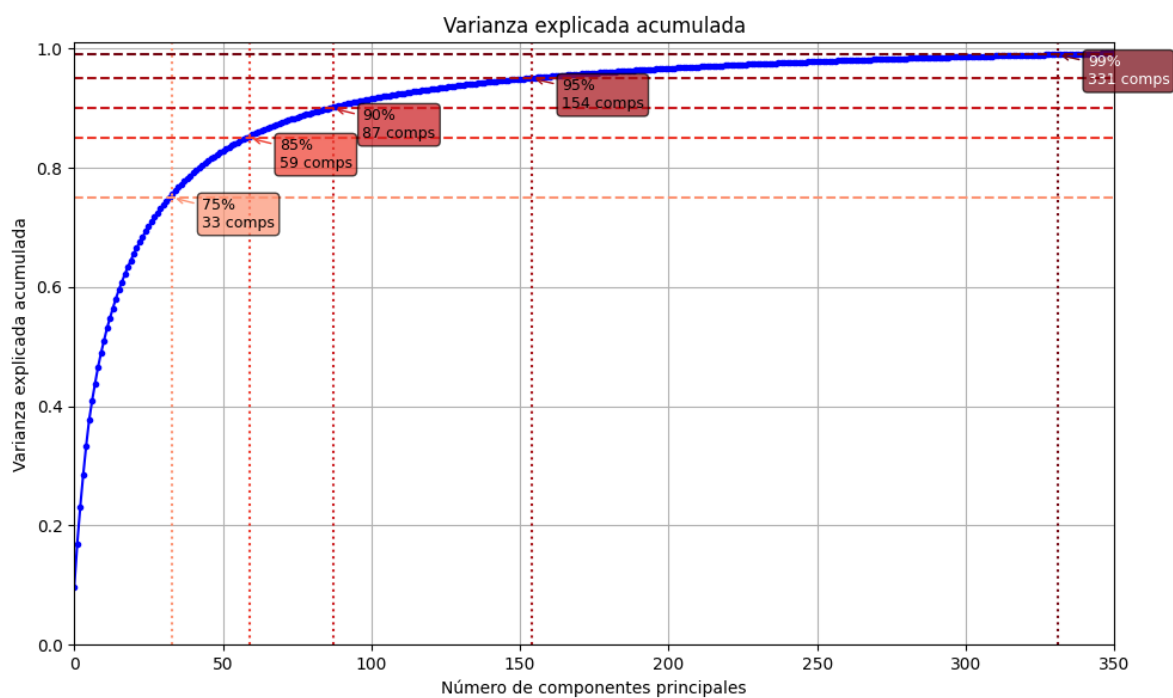


Figura 11: Varianza explicada acumulada en función del número de componentes retenidos en PCA.

A.6. Variación de la reconstrucción en base a la varianza retenida

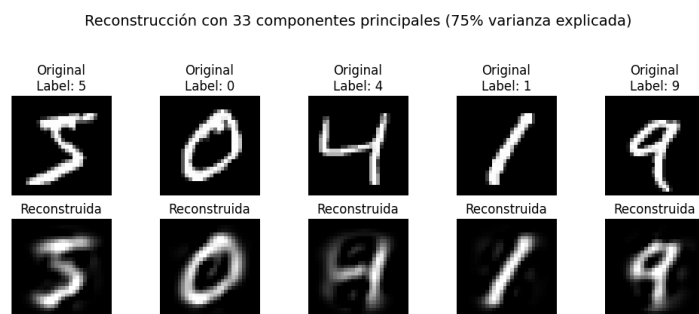


Figura 12: Reconstrucción de imágenes con un 75% de varianza retenida.

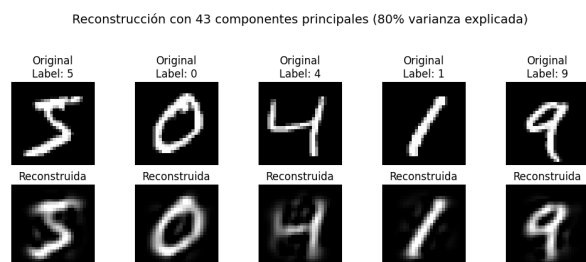


Figura 13: Reconstrucción de imágenes con un 80 % de varianza retenida.

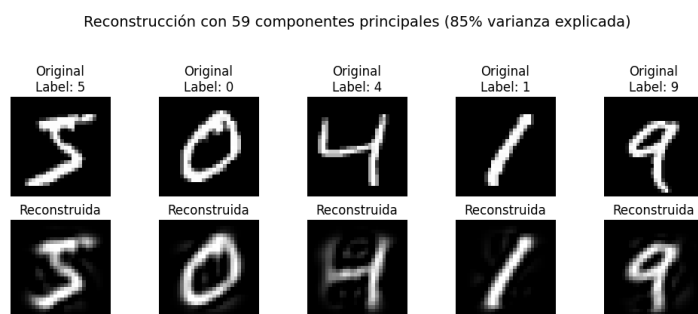


Figura 14: Reconstrucción de imágenes con un 85 % de varianza retenida.

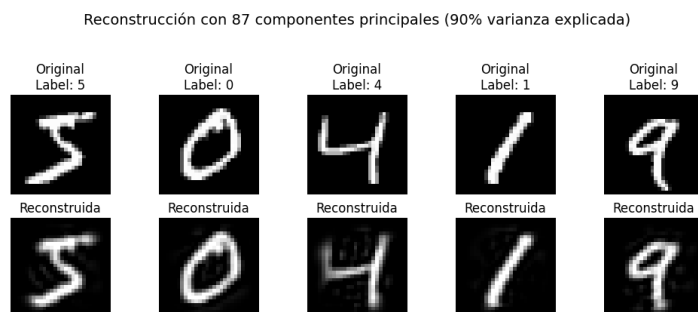


Figura 15: Reconstrucción de imágenes con un 90 % de varianza retenida.

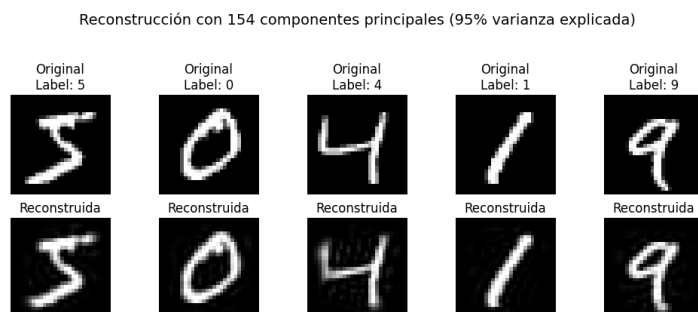


Figura 16: Reconstrucción de imágenes con un 95 % de varianza retenida.

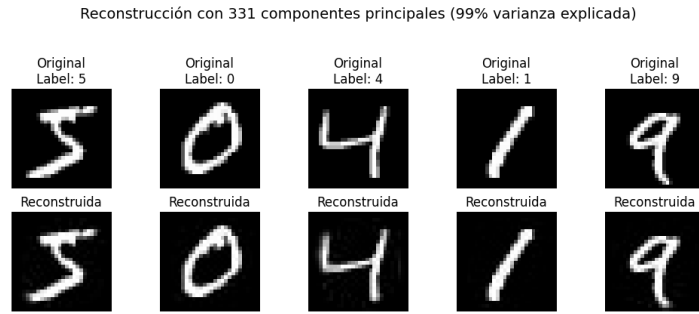


Figura 17: Reconstrucción de imágenes con un 99 % de varianza retenida.

A.7. Representación de las primeras 10 muestras del dataset con PCA



Figura 18: Reconstrucción de imágenes con un 95 % de varianza retenida (10 imágenes).

A.8. Comparación de la calidad de las imágenes reconstruidas de VAE contra PCA



Figura 19: Reconstrucción de imágenes generadas por el VAE.

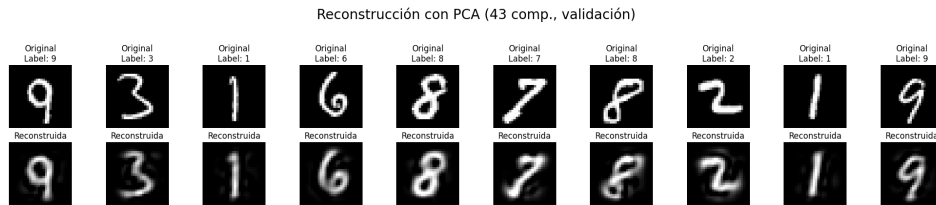


Figura 20: Reconstrucción de imágenes con PCA utilizando 43 componentes (80 % de varianza retenida) sobre el conjunto de validación.